on port 4567 and retry making requests from the alternative UI on port 9999, you'll be able to login. However, if you now try to create a space the request is rejected with a 401 response and you'll end up back at the login page!

> **TIP** You don't need to list the original UI running on port 4567, because this is served from the same origin as the API and won't be subject to CORS checks by the browser.

The reason why the request is blocked is due to another subtle detail when enabling CORS with cookies. In addition to the API returning Access-Control-Allow-Credentials on the response to the login request, the client also needs to tell the browser that it expects credentials on the response. Otherwise the browser will ignore the Set-Cookie header despite what the API says. To allow cookies in the response, the client must set the `credentials` field on the fetch request to `include`. Open the login.js file in your editor and change the fetch request in the login function to the following. Save the file and restart the UI running on port 9999 to test the changes.

```
fetch(apiUrl + '/sessions', {
    method: 'POST',
    credentials: 'include',    #A
    headers: {
        'Content-Type': 'application/json',
        'Authorization': credentials
    }
})
```